CSA06 – DESIGN AND ANALYSIS OF ALGORITHMS
ASSESSMENT 2 (CO2): DEBUGGING AND OPTIMISATION TASK
KRISHA MEENATCHI M (192511338)

Scenario – SOC Alert-Triage Module of 'SentinelOps'

SentinelOps is the Security Operations Centre (SOC) of a cloud-services company. A junior developer wrote a Python module to: (1) sort incoming alerts by severity, highest first, keeping alerts of equal severity in arrival order; (2) check whether a source IP address is on the blocklist; (3) find every position of an attack signature in a log line; and (4) find the two login attempts whose locations are closest to each other, which the SOC uses to flag clustered credential-stuffing attacks. The module passed code review but failed in production: alerts disappeared or were duplicated, blocked IP addresses got through, signatures at the end of a log were missed while false signatures were reported, and the 'closest logins' report flagged logins hundreds of kilometres apart. The module must be fixed before it processes about 50,000 alerts and batches of 8,000 login events per day.

Code Under Investigation (Faulty Module)

```
import math

def sort_alerts(alerts):            # insertion sort by severity, highest first
    for i in range(1, len(alerts)):
        key = alerts[i]
        j = i - 1
        while j > 0 and alerts[j][1] < key[1]:
            alerts[j + 1] = alerts[j]
            j -= 1
        alerts[j] = key
    return alerts

def is_blocked(blocklist, ip):      # sequential search
    for i in range(len(blocklist)):
        if blocklist[i] == ip:
            return False
        else:
            return False

def find_signature(log, pattern):   # brute-force string matching, all occurrences
    n, m = len(log), len(pattern)
    hits = []
    for i in range(n - m):
        for j in range(m):
            if log[i + j] != pattern[j]:
```

```
            break
        if j == m - 1:
            hits.append(i)
    return hits

def closest_logins(events):        # brute-force closest pair of login locations
    min_dist = 0
    pair = None
    for i in range(len(events)):
        for j in range(len(events)):
            d = math.sqrt((events[i][0] - events[j][0]) ** 2 +
                          (events[i][1] - events[j][1]) ** 2)
            if d > min_dist:
                min_dist = d
                pair = (i, j)
    return pair, min_dist
```

Standard Test Dataset

alerts	[("A1", 3), ("A2", 9), ("A3", 5), ("A4", 9), ("A5", 1)]
blocklist & queries	blocklist = ["10.0.0.5", "10.0.0.9", "172.16.4.2"] | queries = ["10.0.0.9", "10.0.0.5", "8.8.8.8"]
log & pattern	log = "FAILFAIXFAIL" | pattern = "FAIL"
events	events = [(2, 3), (12, 30), (40, 50), (5, 1), (12, 10), (3, 4)] # (x, y) coordinates in km
expected targets	Alerts: Sorted high-to-low stably | IP Queries: [True, True, False] | Hits: [0, 8] | Closest: Pair (0, 5) dist ≈ 1.414 km

Task Questions & Assessment Criteria

(a) Reproduce and Identify the Defects (10 Marks)

Run the module on the standard test data. For each function, record the expected output and the actual output, then locate every defect: state the function, the faulty line, and the precise nature of the error. The module contains nine defects, none of which are marked in the code.

(b) Explain Root Causes and Debug Systematically (12 Marks)

For each defect, explain its root cause and how it produces the observed wrong output. Correct the functions one at a time, supporting your reasoning with step-by-step traces (e.g. insertion trace for alerts, sequence search trace, string match alignment table, closest pair distance matrix

trace). Re-test after every correction. Include boundary tests: empty blocklist, pattern at very end of log, pattern longer than log, equal severities, and fewer than two login events.

(c) Optimise the Corrected Module (10 Marks)

Without changing any output, improve the corrected module for the production workload (50,000 alerts, 8,000 login events). Justify whether insertion sort remains suitable for nearly sorted alerts; propose a better approach for repeated blocklist lookups; ensure string matching avoids redundant comparisons (e.g. KMP algorithm); and for closest_logins, eliminate self/reverse pairs, remove square-root calculations, and prune distant pairs.

(d) Analyse Before and After (10 Marks)

Derive the basic operation counts and time complexity of each function before and after optimisation. Measure the distance computations of brute-force and optimised closest_logins for at least three batch sizes between 1,000 and 8,000 events (e.g. N=1000, 4000, 8000), and compare insertion-sort comparisons on nearly sorted vs random alerts. Explain whether empirical measurements agree with theoretical analysis.

(e) Deliver Production-Quality Code (8 Marks)

Submit the final, optimised module with docstrings, meaningful comments, and a comprehensive test harness that checks every function including all boundary cases from (b). Ensure code is structured, robust, and clean.